Principles of Programming Languages: Problem Sheet 3 – Semantic variations

Mike Spivey, Michaelmas Term 2012.
Last revised by Sam Staton, Michaelmas 2017.

1 Show that the monad of failure satisfies the three monad laws, and that the operation *orelse* is associative with unit element *failure*.

2 The exceptions in *FunFail* are the simplest possible, in that an exception does not have any value attached to it. In this exercise, we will implement a slightly more elaborate exception mechnism. Exceptions are raised by calling fail(x), where x is a value that is propagated to the handler. The orelse construct is modified so that the second argument is a function; if the first argument expression raises an exception, then the value attached to it is passed to this function. For example:

```
>>> 1 orelse (lambda (x) x+1);;
--> 1
>>> (fail(3) + 8) orelse (lambda (x) x+1);;
--> 4
>>> fail(3) orelse (lambda (x) fail(x+1));;
*failed (4)*
```

In the first expression, the sub-expression 1 succeeds, so its value becomes the value of the whole expression. In the second expression, the left-hand operand of orelse fails, raising an exception with value 3; this value is used in the exception handler to calculate the value that is returned. In the third example, the exception handler raises another exception.

An implementation of this language can be based on the monad type

$$M\ \alpha = Ok\ \alpha \mid Fail\ Value$$

(a) Show how to define *result* and $\triangleright$ so as to make M into a monad. (You need not check the monad laws.)

(b) Define operations

$$failure :: Value \to M\ \alpha$$
$$orelse :: M\ \alpha \to (Value \to M\ \alpha) \to M\ \alpha$$

on the monad M, suitable for implementing fail and orelse.

(c) Give interpreter clauses for fail and orelse.

(d) Work out what result your interpreter gives for the expression

```
2 orelse fail(3)
```

(Here the right-hand operand of orelse fails, rather than yielding an exception-handling function, unlike the expression 2 orelse (lambda (x) fail(3)), where the exception handler is well-defined but fails when it is invoked.) Would it be possible to give a different result for such expressions by changing the interpreter? □

The next few exercises ask you to investigate the interaction of two language features: assignable variables and exceptions. We will see that there are two different ways to design a language with these two features, and that each way gives a language where programs have a different meaning.

Throughout these exercises, we will use the type Maybe α, defined in the Haskell prelude by

data *Maybe* α = *Just* α | *Nothing*

Like the monad of failure, this type allows us to represent results that may represent successful completion (Just x) or failure (Nothing).

3 The type $M_1\ \alpha$ is defined by

type $M_1\ \alpha = Mem \to Maybe\,(\alpha, Mem)$

(a) Show how to make M_1 into a monad by defining *result* and ▷ appropriately. (You need not check the monad laws.)

(b) Define the following language-specific operations on M_1:

$new :: M_1\ Location$
$get :: Location \to M_1\ Value$
$put :: Location \to Value \to M_1\ ()$

$failure :: M_1\ \alpha$
$orelse :: M_1\ \alpha \to M_1\ \alpha \to M_1\ \alpha$

– in other words, all the operations that are supported by either the monad of memory or the monad of failure.

4 Repeat Exercise 3, but this time using the type $M_2\ \alpha$, defined by

type $M_2\ \alpha = Mem \to (Maybe\ \alpha, Mem)$

5 Now consider two interpreters for a language that combines the features of *FunMem* and *FunFail*. Both are obtained by pasting together all the interpreter clauses (for pure *Fun* plus assignment, sequencing, while and orelse) and primitives (those of pure *Fun* plus new, ! and fail) from the two interpreters *FunMonad* and *FunFail*, but one is based on M_1 and its attendant operations, and the other is based on M_2.

(a) Explain in words the difference between the types $M_1\ \alpha$ and $M_2\ \alpha$.

(b) Explain what difference this makes in the languages implemented by the two interpreters. Which would be easier to implement efficiently?

(c) Write a program in the extended *Fun* language that gives a different result with the two interpreters. □

Many languages, such as C, Java and Scala, have 'assignable' aka 'mutable variables': the values assigned to identifiers can be changed during execution. The language FunC shares the syntax of Fun, but identifiers denote assignable variables, and the operation of fetching the contents of a variable is implicit.

Thus in FunC, the following expression is legal:

```
let val x = 3 in x := x + 1; x
```

Here the declaration val x = 3 *creates an* assignable *variable* x*, and the expression* x + 1 *implicitly fetches the contents of this variable. The same program in Fun with memory would have to be written as*

```
let val x = new() in x := 3; x := !x + 1; !x
```

making both the creation of the assignable variable and the accessing of its contents into explicit operations.

Here is the factorial function re-implemented in this language:

3A ⟨ File facref.fun 3A ⟩ ≡

```
rec fac(n) =
  let val k = n in
  let val r = 1 in
  while k > 0 do
    (r := r * k; k := k - 1);
  r;;
```

For now, we consider FunC with two restrictions: the left-hand side of an assignment must be a variable, and in a function call $f(e_1, \ldots, e_n)$, the function f must be a variable.

We can translate FunC expressions e into FunMem expressions $e^\sharp$ as follows.

- *each assignment x := e in FunC is translated to x := $e^\sharp$ in FunMem. In other words $(x := e)^\sharp = (x := e^\sharp)$.*
- For each variable f in the initial environment, i.e. the built-in functions, the arguments are passed by value: a function call $f(e_1 \ldots e_n)$ in *FunC* is translated to $f(e_1^\sharp, \ldots, e_n^\sharp)$ in *FunMem*. In other words, $(f(e_1 \ldots e_n))^\sharp = f(e_1^\sharp, \ldots, e_n^\sharp)$ for built-in f.
- For each variable f not in the initial environment, i.e. a user defined function, the arguments are passed by reference. This means that a function call $f(e_1 \ldots e_n)$ in *FunC* is translated to $f(ref(e_1^\sharp), \ldots, ref(e_n^\sharp))$ in *FunMem* where

  ```
  val ref(a)=let val x=new() in x:=a;x
  ```

  In other words, $(f(e_1 \ldots e_n))^\sharp = f(ref(e_1^\sharp), \ldots, ref(e_n^\sharp))$ when f is not built-in.
- A value definition val *x*=*e* in *FunC* is translated to a reference definition val *x*=ref($e^\sharp$) in *FunMem*.
- All other expression structure is preserved by the translation.

6 Implement this translation as Haskell functions

etranslate :: *Expr* → *Expr*
dtranslate :: *Defn* → *Defn*

```
void swap(int *a, int *b) {
    int t = *a;
    *a = *b; *b = t;
}

int main() {
    int x = 3, y = 4;
    swap(&x, &y);
    printf("%d %d\n", x, y);
    return 0;
}
```

Figure 1: *Swap written in C.*

Use these functions to modify the *FunMem* definitional interpreter, and test your new *FunC* definitional interpreter with the programs above. □

In C (but not in Scala or Java) the location of any variable can be obtained as a pointer value that can later be used to access or update the variable.

We can mimic this feature by adding two new forms of expression to the syntax of our language FunC:

data *Expr* = ...
| *Address Expr* — $\&e_1$
| *Contents Expr* — $*e_1$

The character '' is also used for the multiplication operator, but we rely on the parser to deal with this potential confusion.*

*The idea is that if x is a variable, then &x is its location as a value; and if a is such a value, then *a is the contents of the location. We are going to allow * to be used on the left-hand side of an assignment, so that *a := e changes the same cell whose contents are fetched by the expression *a.*

These constructs can be used to get the effect of parameters passed by reference in a language like C, where all parameters are in fact passed by value. Figure 1 shows a C program where the function swap *exchanges the values of its two arguments, which are passed as pointers.*[1]

7 This question is about the aspects of C described above.

(a) Rewrite the C function swap in *FunMem*, which has locations as expressible values and explicit dereferencing. Also write an example script based on the C main function.

(b) Extend your translation from Question 6 to handle & and *, and hence obtain a definitional interpreter for this extended version of *FunC*.

(c) Rewrite the C function swap in *FunC*, so that it works as follows:

```
>>> val x = 3;;
--- x = 3
>>> val y = 4;;
```

[1] The parameter declaration int *a, int *b simply declares two formal parameters a and b with the type int * or 'pointer to integer'.

```
--- y = 4
>>> swap(&x, &y);;
--> 3
>>> x:y:nil;;
--> [4, 3]
```

(If you implement things exactly the same as me, then swap(&x, &y) will return the initial value of x.)

(d) Show that if x is an assignable variable, then the expression $*(\&x)$ yields the same value as the expression x.

(e) Is it necessarily true that $\&(*x)$ yields the same value as x? Justify your answer.

Be ready to explain the differences in the way addressing operators like &, $*$ and ! are used in the different languages. □

8 Here is a naive definition of $fib :: Int \to Int$:

$$\begin{array}{l} fib\ n = \\ \quad \textbf{if}\ n \leq 1\ \textbf{then}\ n\ \textbf{else}\ fib\,(n-1) + fib\,(n-2) \end{array}$$

Convert this definition to CPS form.

9 The monad of continuations is defined by

$$\textbf{type}\ M\ \alpha = (\alpha \to Answer) \to Answer$$

$$\begin{array}{l} result :: \alpha \to M\ \alpha \\ \quad (\equiv \alpha \to (\alpha \to Answer) \to Answer) \\ result\ x\ k = k\ x \end{array}$$

$$\begin{array}{l} (\triangleright) :: M\ \alpha \to (\alpha \to M\ \beta) \to M\ \beta \\ \quad (\equiv ((\alpha \to Answer) \to Answer) \to \\ \qquad\quad (\alpha \to (\beta \to Answer) \to Answer) \to \\ \qquad\quad (\beta \to Answer) \to Answer) \\ (xm \triangleright f)\ k = xm\,(\lambda\, x \to f\ x\ k) \end{array}$$

Show that this definition satisfies the three monad laws.

10 (Optional)

(a) Implement for *FunC* the primitive new() that we had in *FunMem*, and compare it with the function malloc in C or the procedure NEW in Oberon. What does this say about the nature of variables in our language *FunMem*?

(b) What is the difference between the following programs in *FunC*?

```
let val x = new() in x := 3; x;;
```

```
let val x = new() in *x := 3; x;;
```

(c) What is wrong with this C program?

```
int *allocate() {
  int space;
  return &space;
}
```

```
int main() {
  int *p = allocate();

  *p = 3; printf("%d\n", *p);

  return 0;
}
```

11 (Optional) Show how (in addition) to provide arrays as values in our dialect of FunC, so that the declaration,

array $a[n]$

declares a to be an array of n elements, each of them (let us say) initialised with *Nil*. Subsequently, we should be able to refer to elements of a with the expression $a[i]$, and to set them with the assignment $a[i] := x$. How should arrays interact with the *Address* and *Contents* operators we introduced earlier?